Privacy Guardians

Maximally Private Payments on Ethereum, within the Law.

by Leo Glisic

(written in collaboration with Fable)

July 2026

Abstract

This paper presents Privacy Guardians, a private payment network built on Ethereum. Payment systems available today require their users to accept one of three trade-offs. Traditional payment rails expose each transaction to a chain of intermediaries (the merchant's processor, the acquiring bank, the card network, the issuing bank, and the data and fraud vendors behind them), and every account within them can be suspended by its operator. Public blockchains remove the operator's ability to suspend a self-custodied account, but they publish every balance and every transaction permanently. Anonymity-focused systems provide strong privacy but offer no lawful-access mechanism, and as a result they have been sanctioned or restricted in many jurisdictions, preventing strong network effects.

The design presented here is intended to provide privacy, censorship resistance, and regulatory compliance simultaneously. Users transact through Privacy Guardians of their choosing, which may be regulated institutions, merchant networks, or, where law permits, decentralized operators. A user's own Guardian has complete visibility into that user's account; no other party, including other Guardians, can observe balances, amounts, or counterparties. Guardians operate the network's private accounting but never take custody of funds, which are held in a neutral settlement contract on Ethereum. Each Guardian's books are proven consistent with the funds behind them, cryptographically, at every settlement, so misstated books cannot settle. Any user can withdraw to Ethereum, or move to a different Guardian, without their current Guardian's cooperation. Reserves are held in yield-bearing form, and the resulting yield funds the network: in the example figures used throughout, half compensates Guardians and half funds privacy insurance, including a standing bounty (the honeypot) that gives anyone who obtains a Guardian's dataset a standing incentive to reveal the theft; once the theft is proven, compensation is automatic.

Compliance is structured jurisdictionally. Each government determines which Guardians may serve its residents, lawful process is answered by a user's Guardian in the same manner as by a bank, and withdrawals settle to the public ledger, so funds can always leave the network and every withdrawal is publicly visible. The design is intended to make available, within each jurisdiction, the maximum financial privacy that its law permits, which in most jurisdictions is considerably more than existing payment rails provide. The protocol defines no investment or governance token (user balances constitute a fully reserve-backed private stablecoin), and the design is open-sourced for anyone to implement; the author is not building it himself. This document is a white paper: it specifies the architecture and the incentive structure, and defers engineering detail to a subsequent document.

1. Introduction

1.1 The migration of payments on-chain

Stablecoin settlement volume has grown into the hundreds of billions of dollars, and the largest institutions in the payments industry are now building on-chain payment rails of their own. We take the direction as settled: a meaningful share of the world's everyday payments will move onto public blockchains. The remaining question is which properties the resulting networks will have. Payment systems concentrate around a small number of winners through network effects, and the properties of

those winners tend to persist; we would therefore expect the properties of on-chain payment networks to be determined early and to be difficult to change afterward.

This paper concerns two of those properties, privacy and censorship resistance. Both are largely absent from the on-chain payment systems being built today, and the design presented here is intended to show that both can be provided without any reduction in regulatory compliance.

1.2 Questions every payment system must answer

Two questions determine much of the character of any payment system: which parties can observe a payment, and which parties can suspend an account.

Traditional finance provides weak answers to both. An ordinary card payment is observed by the merchant's processor, the acquiring bank, the card network, the issuing bank, and the analytics and fraud vendors behind them. Each records the payer, the payee, the amount, and the time; each retains the record for years; and each represents a potential point of breach. Privacy in this system is a policy commitment made in a terms-of-service document rather than a property of the architecture. With respect to the second question, any account can be suspended by the institutions that operate it. In most cases that power is exercised lawfully; the relevant observation is that the power exists, is concentrated, and is not visible to the user until it is exercised.

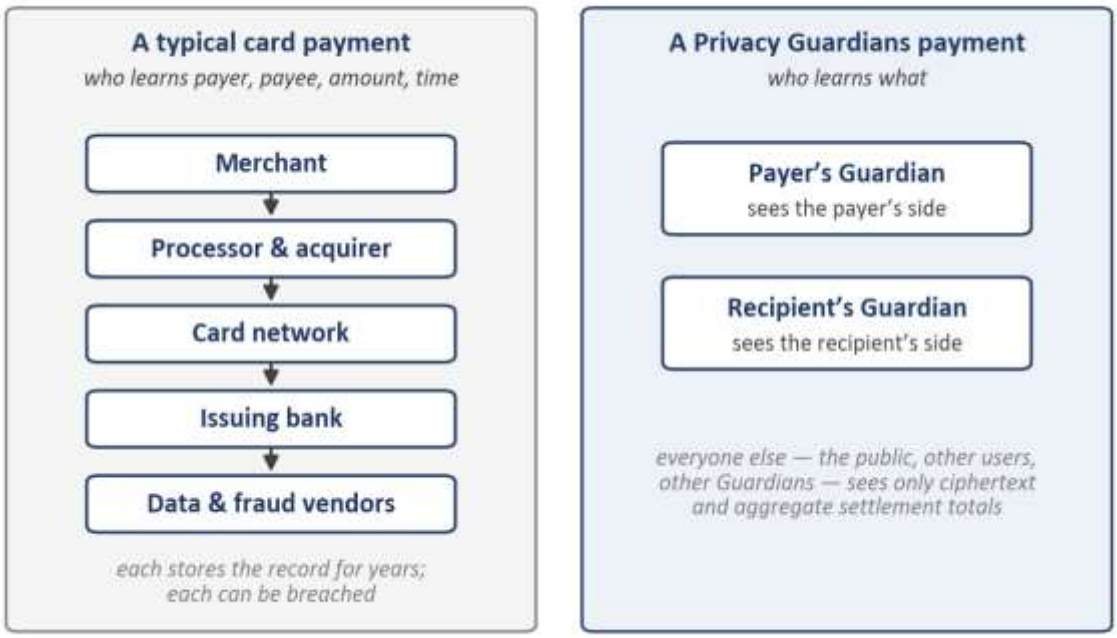

Figure 1: The parties able to observe an ordinary payment today, and the parties able to observe a payment on the Privacy Guardians network.

Public blockchains invert both answers. No party can freeze a self-custodied account, which resolves the second question. However, the first question receives a weaker answer than before: every balance and every transfer is published, permanently, to all observers. A salary paid on Ethereum today is public

information, and a merchant accepting payment on a public chain reveals its complete revenue history, in real time, to its competitors.

A third family of systems, the anonymity technologies, conceals activity from all observers. These systems resolve both questions in the user's favor, but they provide no lawful-access mechanism, and governments have responded by sanctioning and restricting them. Whatever position one takes on those decisions, the practical consequence is that anonymity systems have not carried, and in our view cannot carry, the everyday payments of people subject to functioning legal systems.

In other words, the systems available today offer three combinations: broad institutional visibility with consumer protections, full public exposure with self-custody, or strong privacy without legal standing.

1.3 The design goal

The goal of the design presented here is to answer both questions well at the same time, within existing law. Concretely, the network must satisfy four properties at once: it must be decentralized; it must be censorship-resistant, a property which, on Ethereum, follows from being non-custodial; it must provide more privacy than traditional finance provides today; and it must be built for full compliance in every country where it operates. Sections 3 through 8 specify a design that we believe satisfies all four.

The motivation for this work is twofold. First, we would like to see Ethereum widely adopted, and we consider everyday payments the largest unmet use case standing between Ethereum and ordinary users. Second, we consider the erosion of financial privacy to be an architectural problem, and one that a payments layer designed for privacy can address directly. A payments network that users prefer on its own merits (immediate, inexpensive at the point of use, and private) would serve both purposes.

2. Privacy Within the Law

2.1 Privacy as an option

The design treats privacy as an option to be exercised rather than a requirement to be imposed. Some users will exchange privacy for lower fees, for stronger consumer protections, or for familiarity, and we would expect many to do so. The design accommodates that choice. A protocol cannot make users value privacy more than they do; it can only ensure that the private option exists, is competitive, and is fairly priced. In other words, the design standard is that each user receives the amount of privacy they choose, and that the most private option available is a competitive product.

2.2 The maximum varies by country

The amount of financial privacy available to a person is ultimately set by their country's law, which determines which records must exist, which parties may see them, and under what process. The network does not exist to change those laws, and it does not need to. Its purpose is to deliver the full amount of privacy that the citizens of each country have already secured under their own legal system.

It is worth noting how large that amount is. In most countries, the lawful maximum is considerably more private than what the incumbent rails deliver. The gap exists not because the law forbids private payments, but because payment data is commercially valuable to the intermediaries that operate the

incumbent systems, and no alternative has been built. Governments retain every power they currently hold, including warrants, subpoenas, and lawful process generally (§7). What changes is the set of other parties able to observe a user's payments: under this design, that set is the smallest it has to be.

2.3 Privacy Guardians

The mechanism is a market. Every user transacts through a Privacy Guardian, the party that processes their payments, maintains their account, and is accountable for it. Any entity can operate a Guardian: a regulated bank, a licensed financial technology firm, a merchant network, a telecommunications company, or, where law permits, a fully decentralized protocol, in which case users need not trust any institution. There is deliberately no default; each user selects, from among the Guardians authorized in their jurisdiction, the one whose protections and pricing they prefer.

Each government, in turn, determines which Guardians may serve its residents, which is the same authority it exercises over any regulated business. A government that objects to a particular Guardian can prohibit that Guardian from operating. We would expect no government to need to prohibit the protocol itself, because the protocol takes no positions a government could object to: every policy that governments regulate (identity standards, records, disclosures) is implemented by Guardians, within national law (§7).

2.4 Alignment by incentives

The arrangements described above could be attempted with a terms-of-service agreement and a trusted operator, and they would then depend on that operator's continued good behavior. The premise of this design is that such promises are durable only when they are enforced by mechanisms stronger than reputation. There are three such mechanisms, and they organize the remainder of the paper.

- **Non-custody is enforced by contract.** Guardians never hold user funds and cannot freeze or trap them. Any user can exit, or switch Guardians, without any party's cooperation (§3).
- **Honesty is enforced by proof:** a Guardian's books must be shown consistent with the funds behind them, cryptographically, at every settlement. Books that do not match the funds cannot settle (§3).
- **Privacy is enforced by price:** a Guardian that leaks its users' data faces detection rewarded by a standing bounty, automatic compensation to every affected user, and the loss of its own stake, with verification and payment executed by smart contract (§4, §5).

The remainder of the paper describes the machinery. Section 3 specifies the payment protocol: how funds enter without revealing a Guardian relationship, how payments remain private, how settlement is proven, and how users exit without permission. Section 4 describes the economics that fund the network and its insurance. Section 5 specifies the honeypot, the mechanism that creates a standing incentive for any data breach to be revealed, and compensates it automatically once proven. Section 6 describes the user experience, including consumer protections. Section 7 sets out the compliance model, and Section 8 the governance posture and directions for further work.

3. The Encrypted Protocol

This section specifies the network's core machinery: how a user joins and selects a Guardian without that relationship being publicly visible; how payments execute so that no observer of the chain can reconstruct who paid whom, or how much; how value settles between Guardians; and how any user can exit, without the permission of any party, at any time.

Two terms recur throughout. The Depository is the protocol's single settlement contract on Ethereum mainnet; it holds every deposited dollar. A Guardian is a service operator, in most jurisdictions a regulated one (§7), that a user chooses as their counterparty for privacy, service, and compliance. Guardians operate the network's private accounting, and they do not hold user funds at any point.

3.1 What the protocol must achieve

The design must satisfy five properties simultaneously. Each property is straightforward to state in isolation; the substance of this section lies in satisfying all five at once.

P1. Private association. When a user joins the network and selects a Guardian, only the user and that Guardian learn of the relationship. Nothing on the chain associates the two.

P2. Private activity. Balances, payment amounts, and counterparties are invisible to the public, to other users, and to every Guardian other than the user's own. The aggregate amounts that necessarily appear on the chain must not reveal individual payments.

P3. Non-custody. Guardians never hold or control user funds. All funds reside in the Depository, a neutral contract that no Guardian, and no consortium of Guardians, can use to trap value.

P4. Permissionless departure. A user can always withdraw to Ethereum, or move to a different Guardian, without their current Guardian's cooperation or consent.

P5. Provable totals. The total value under each Guardian's guardianship is publicly provable at all times, while revealing nothing about any individual account.

A sixth property is one of service rather than protocol: payments must appear immediate to the user, even though the canonical ledger settles periodically. The mechanism that reconciles an instant payment experience with periodic settlement recurs throughout this section.

One boundary of the privacy model should be noted at the outset. A user's own Guardian sees that user's account completely: identity (to whatever standard its jurisdiction requires), balance, and every payment it processes. This visibility is a deliberate design decision rather than an information leak. The network's privacy claim is privacy from the world, paired with accountability to a single party the user chose, and that party can answer lawful process the way a bank can, within its own jurisdiction's rules (§7). This accountability distinguishes the network from anonymity systems and allows for mass adoption.

3.2 Components

Four components make up the network.

- **The Depository.** A single settlement contract on Ethereum mainnet. It holds the network's reserves; maintains one public running total per Guardian (that Guardian's "bucket"); holds newly arrived deposits in a pending pool until they are claimed; stores one commitment (the state root) that summarizes every private account balance in the network; verifies the zero-knowledge proofs that advance that commitment; and processes exits. The Depository is designed to run without discretionary control; its few parameters and its upgrade posture are treated in §8.
- **The message board.** A high-throughput, low-cost data plane on which all protocol messages are posted: encrypted payment intents, cross-Guardian messages, receipts and attestations, and the per-epoch record of changed account commitments. The design requires four properties of this venue (canonical ordering, guaranteed data availability, forced inclusion so that no operator can silently censor a message, and low per-byte cost) and deliberately does not require a particular venue. A general-purpose rollup, an application-specific rollup, or Ethereum blob space could all satisfy these requirements.
- **Guardians.** Service operators, each running private books for its users, an encryption endpoint for messages addressed to it, and a prover that periodically demonstrates, in zero knowledge, that its books are exactly consistent with the canonical message record. Guardians hold working capital and carry insurance (§4); they may be banks, licensed fintechs, merchant collectives, or, where permitted, decentralized operators (§7).
- **Users.** Each user runs a client (typically a mobile payment app) that holds a single seed. All other information the client requires can be reconstructed from public data (§3.3).

The network operates on two layers. The experience layer is the Guardians' private books: when a user pays, their Guardian confirms in under a second. The canonical layer is the encrypted ledger committed to by the state root, which advances once per epoch, an interval expected to range from minutes to about an hour (§3.7). The books serve as a cache, while the proven ledger is the canonical record. This division is the same one card networks have used for decades between instant authorization at the terminal and clearing and settlement completed later. The differences are that clearing here completes in minutes (card clearing takes days), and that correctness is checked by a mathematical proof rather than by an audit.

The Depository accepts deposits in the major dollar stablecoins and holds its reserves as a diversified basket of yield-bearing dollar instruments (§4); balances are denominated in dollars and redeemable one-for-one (in effect a private, fully reserve-backed stablecoin; the network issues no investment or governance token). Examples throughout use USDC as the deposit asset.

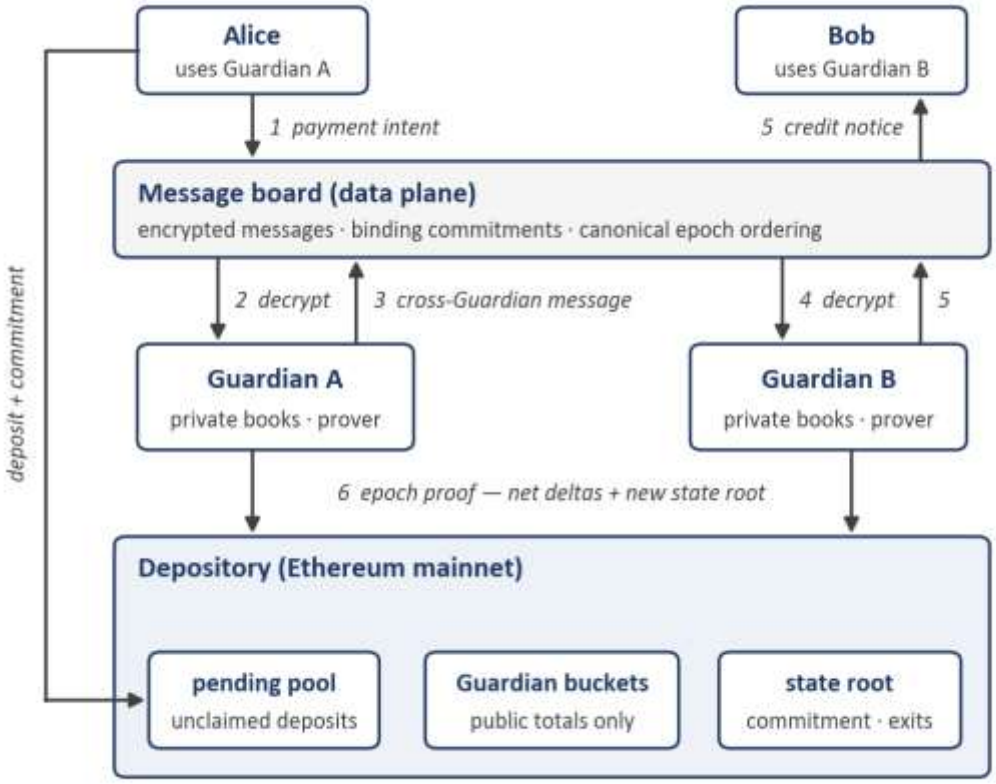

Figure 2: The four components, with a cross-Guardian payment traced through them (steps 1–6 are described in §3.6 and §3.7).

3.3 Accounts, keys, and statelessness

From one seed, a user's client derives a signing keypair, which authorizes payments and exits, and an encryption keypair, which receives private messages. The account identifier is a hash of the public keys: a pseudonym. The only on-chain link between a user's Ethereum address and their account is the deposit transaction itself, and the deposit, as §3.5 describes, names no Guardian and no account.

In the canonical ledger, an account is a single entry (a "leaf") containing a hiding commitment to the tuple (account key, balance, nonce, security-policy hash). A hiding commitment reveals nothing about its contents, which is why the protocol can publish these entries freely; the policy hash pins the user's chosen exit rules (§3.8) so that even the user's Guardian cannot weaken them unilaterally.

Every message that affects a user's account (intents, credit notices, receipts, balance attestations) is posted to the message board encrypted to the user, and all randomness the client uses is derived from the seed. A client holding only the seed can therefore reconstruct its keys, its full history, its receipts, and its current ledger entry from public data. As a result, the loss of a device costs only the time required to restore, and there is no file whose loss forfeits funds. To keep repeated payments to the same person unlinkable, recipients share payment handles from which a sender derives a fresh one-time tag for each payment; routing information inside a handle is readable by Guardians, not by other users.

3.4 The partitioned private ledger

The canonical ledger is partitioned into one subtree per Guardian, each containing the ledger entries of that Guardian's users; it is neither a single shared tree nor any one Guardian's database. The state root stored in the Depository commits to all subtrees at once. Each Guardian knows the plaintext contents of its own subtree (its users' balances) and nothing about any other Guardian's.

Each epoch, the commitments that changed are published to the message board. Because they are hiding commitments, this publication reveals nothing beyond an approximate count of active accounts; what it provides is data availability, the raw material any user needs to prove their own balance against the state root without their Guardian's help. This capability is what allows the exits of §3.8 to proceed without any Guardian's cooperation.

The partition also turns the network's solvency condition into an arithmetic identity. Every epoch proof is required to demonstrate:

$$\textit{for every Guardian } G: \quad \textit{bucket}(G) \;=\; \Sigma \textit{ balances in } G\text{'s subtree}$$

The left side is public (the Depository's running total for that Guardian). The right side is private. The proof shows they are equal without opening either. As a result, a Guardian cannot credit its users, in aggregate, with more than its bucket holds. In other words, an epoch containing fractional reserve is unprovable and cannot settle, so the condition is prevented at settlement rather than detected after the fact. This is property P5, delivered continuously and by construction: anyone can read every Guardian's total under guardianship off the Depository at any time, with a proof behind it, and no individual account revealed. The totals serve purposes well beyond transparency (insurers price against them, and counterpart Guardians set credit limits against them; §4). Within the protocol itself, they serve the narrower but critical purpose of ensuring that dishonest books cannot settle.

3.5 Joining: deposits that name no Guardian

A deposit that named its Guardian would publish a relationship the network is built to keep private (P1). Joining therefore proceeds in five steps.

1. Handshake. The user contacts their chosen Guardian (through its own app or through any compatible payment app) and completes whatever onboarding that Guardian's jurisdiction requires (§7). This happens before any funds move, so a Guardian never receives a deposit from a user it would decline.
2. Deposit. The user sends funds, for example 500 USDC (illustrative), to the Depository, attaching a commitment:

$$C \;=\; H(\textit{ account id} \;\|\; \textit{Guardian id} \;\|\; \textit{salt}\ )$$

(H is a hash function; ‖ denotes concatenation; the salt, derived from the user's seed, prevents guessing.) Publicly, an address deposited 500 USDC into the network. No Guardian is named, no account is visible, and the funds rest in the Depository's pending pool. The deposit is locked (not refundable by the depositor) for a lock window T_lock, a protocol parameter on the order of a day.

3. Service begins immediately. The user has already sent their Guardian the commitment's opening during the handshake. The Guardian verifies that C names it (which means no other Guardian can ever

claim this deposit), and the lock means the depositor cannot withdraw it before T_lock expires. The Guardian's claim is therefore exclusive and guaranteed. On that guarantee it opens the account on its books at once, issues a signed receipt, and the user can begin paying within seconds of depositing.

4. Claim, performed later and alongside others. At a moment of its choosing within the lock window, the Guardian claims the deposit inside a routine epoch proof. The proof demonstrates that some set of pending deposits carries valid commitments naming this Guardian and that their sum is correct, without identifying which deposits. The sum folds into the Guardian's single net settlement figure for that epoch (§3.7), so not even the subtotal of claimed deposits appears separately.
5. Refund path. A deposit still unclaimed when the lock expires becomes refundable by the depositor, permissionlessly. As a result, neither entry into the network nor exit from it requires any party's permission.

If the user spends before their deposit has been claimed, the canonical debit cannot come from their ledger entry, because that entry does not exist yet. It comes instead from the Guardian's working-capital account, an ordinary entry in the Guardian's own subtree funded with its own money. When the Guardian later claims the deposit, the same epoch proof creates the user's entry and repays the working-capital account internally, a movement inside one subtree that is visible nowhere. A Guardian that claims in the very next epoch needs no advance at all; the advance exists so that claiming can wait for a batch of other deposits to form. The capital this requires scales with what new users spend in their first hours rather than with what they deposit; a Guardian onboarding millions of dollars a day would be expected to need working capital orders of magnitude smaller (all figures illustrative).

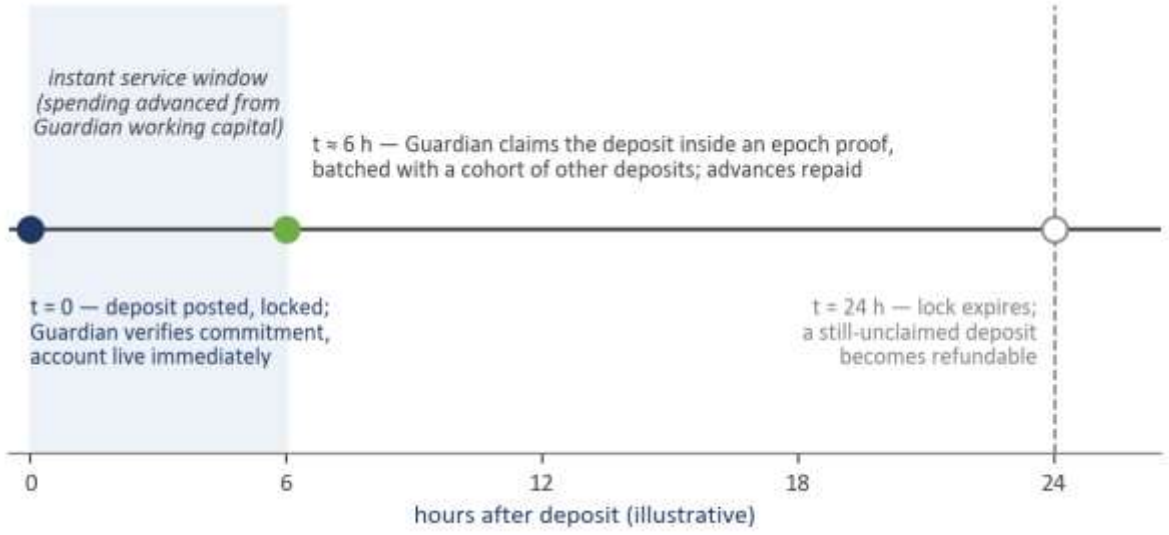

Figure 3: Claim-ahead onboarding. Service begins immediately; the claim is deferred until a batch forms; the lock guarantees the Guardian its claim in the interim.

The privacy of the assignment should be characterized precisely. An observer sees deposits enter the pending pool and sees each Guardian's bucket grow at settlements; the anonymity of the deposit-to-Guardian assignment derives from the set of deposits claimed alongside it. The size of that set depends on how busy the network is, and a network with lower activity provides a smaller one. Guardians can strengthen the anonymity set where needed (minimum claim-batch sizes, randomized claim delays,

patient claiming for users who prefer it), and the protocol leaves the exact policy open. §3.10 treats what the chain shows, and does not show, in full.

3.6 Paying: intents, receipts, and crossing Guardians

A payment begins as an intent: recipient tag, amount, asset, a per-account nonce, and an optional memo, signed with the payer's account key and encrypted to the payer's own Guardian. The client posts it to the message board along with a binding commitment to its core fields. The Guardian that decrypts an intent can neither alter it nor invent one: the settlement circuits verify the user's signature under the commitment, reject any repeated nonce, and reject any account whose balance would go negative.

When payer and payee share a Guardian, the payment does not touch public state at all. The Guardian decrypts, updates both accounts on its books, and issues two signed receipts: the payer's shows the new balance and nonce; the payee's shows the credit. Receipts play a substantial role in the design: every confirmation a Guardian issues is a signed, dated instrument, and it is these instruments that make within-epoch balances enforceable claims and insurable exposures (§4). At the next epoch the Guardian's proof applies the intent to its subtree, and the canonical ledger is brought into agreement with the books.

When the payee is served by a different Guardian, the intent carries a nested encryption: inside the envelope addressed to the payer's Guardian A sits a payload only the payee's Guardian B can read, containing the credit's details. Guardian A debits the payer on its books and posts a cross-Guardian message to the board, addressed to B, bound by the same commitment, and signed by A. Guardian B decrypts, credits the payee on its books instantly, and posts a credit notice encrypted to the payee (a memo from the payer, encrypted end-to-end, can accompany the notice; Guardian B learns the amount and that the payment came through Guardian A, but by default not from whom. Jurisdictions that require originator information for large transfers can mandate an encrypted originator payload readable by the receiving Guardian; the protocol carries the slot and leaves the requirement to §7).

The two sides are tied by the binding commitment: the sending Guardian's epoch proof must consume it as a debit, and the receiving Guardian's as a credit, each exactly once. As a result, value cannot be created, destroyed, or double-counted between subtrees; a movement of value between subtrees requires the payer's signature at the root of the chain of evidence.

Payments to recipients outside the network operate as follows. The payer sends a payout instruction to their Guardian, which pays the recipient's Ethereum address immediately from its own external working capital and posts the corresponding commitment to the board; at the next settlement the payer's account is debited and the Guardian reimbursed from its bucket. The recipient receives an ordinary transfer and need not know the network exists; the payer's account, balance, and Guardian remain private.

3.7 Settling: epochs, netting, and what "final" means

An epoch is the interval between settlements. Its boundary is defined by the message board itself, as an unambiguous slice of the canonical message sequence: every message inside the slice is in the epoch, everything outside is not, and no Guardian chooses which messages count. The completeness rule is central to this construction: a Guardian's epoch proof must consume every message it signed within that epoch, and it cannot selectively omit any. As a result, a signed cross-Guardian message is self-executing:

once Guardian A signs it, the protocol itself will collect it from A's subtree at settlement, whether or not A remains cooperative.

Settlement is parallel by construction. Each Guardian proves its own subtree transition (deposits claimed, intents applied, cross-messages consumed, exits honored, the solvency identity of §3.4 intact). A permissionless aggregation step combines the subtree proofs, checks cross-Guardian consistency, and submits one proof to the Depository. The Depository verifies it, adopts the new state root, and applies a single net figure to each Guardian's bucket. Across all Guardians the figures sum to zero. The public record of an epoch is that list of net figures; it does not include bilateral flows, payment counts, or individual transaction amounts. A Guardian that cannot produce a valid transition (an outage, a lost key, or books that exceed what its bucket can prove) fails to settle; after a bounded number of missed epochs it is delinquent and its subtree freezes at the last proven root (§3.8, §3.11). In other words, a failure halts and exposes the failing Guardian rather than resulting in the unobserved creation of funds.

It is worth being precise about what "final" means in this context. For the recipient of a payment, finality arrives the moment their Guardian's signed receipt does, less than one second after the intent was posted. The receipt is an enforceable instrument, the sending Guardian's message is self-executing at settlement, and over-issuance is unprovable. The residual risk (a Guardian failing mid-epoch with obligations outstanding) is borne by the failing Guardian itself: its working capital is held in the Depository and is forfeitable against unsettled obligations, and the adaptive settlement cadence described below keeps each Guardian's unsettled position within that collateral. In other words, a Guardian that credits an incoming payment at confirmation is not extending uncollateralized credit to the sending Guardian, and an implementation may enforce this bound as a strict protocol rule. As with card networks, users and merchants experience finality at confirmation; here the interval to settlement is minutes rather than days.

The cadence itself is adaptive. An epoch closes at the earlier of a clock and any Guardian's unsettled exposure crossing a threshold tied to its capital. Quiet periods settle at an unhurried pace, while a surge of volume triggers settlement within minutes. We would expect operators to keep epochs short, as shorter epochs mean smaller advances and smaller exposures.

3.8 Leaving: exits and Guardian changes

Permissionless exit is the network's neutrality backstop. However, precisely because it moves funds out of the network's protections, it is also the natural target for theft and coercion. The protocol therefore allows each user to select an exit policy, pinned by the policy hash in their ledger entry: withdrawal delays tiered by amount; optional approver sets (family, a service, possibly the user's own Guardian) with thresholds by tier; and a hard-exit backstop, a long-delay path requiring no approvers at all, so that a user abandoned by every counterparty can still leave. Policy changes take effect slowly, so that an attacker who compromises a device cannot first remove its protections. The full policy space, and the reversibility window for in-network payments, are treated in §6.

Exits come in two forms. A cooperative exit is a ticket processed through the user's Guardian: policy checks, inclusion in the next epoch proof, payment from the Guardian's bucket after the delay. A unilateral exit requires nothing from the Guardian: using the published commitments of §3.4, the user proves in zero knowledge that a ledger entry under the current state root opens to their key and balance, signs with their account key, and is paid by the Depository after the same policy delay. If the Guardian is delinquent and its subtree frozen, exits proceed against the frozen root. In every case an exit finalizes against the

newest proven state, after a delay no shorter than one epoch plus the dispute interval, so that every in-flight payment lands in a root before funds leave.

Changing Guardians uses the same mechanism. Cooperatively, the user's entry moves from one subtree to another inside a routine epoch proof, folded into the net settlement figures: an observer cannot determine that a switch occurred, nor which account was involved. Unilaterally (against a hostile or non-operational Guardian), the user exits without cooperation and redirects the proceeds into the pending pool under a fresh commitment naming the new Guardian, rejoining as §3.5 describes, under a new pseudonym if desired.

Two operations should be distinguished here. A withdrawal to Ethereum is necessarily public (an address and an amount), because it moves funds from the private ledger onto the transparent chain; the network therefore guarantees that a user can depart at any time, and every withdrawal is visible when it occurs. A Guardian change (described earlier in this section) moves an account between Guardians without any funds leaving the network; it is not publicly visible, and the account remains under a Guardian's compliance regime throughout. This distinction is relevant to the compliance analysis in §7.

3.9 A payment, end to end

To illustrate the mechanics described above, this section traces a single payment from deposit to settlement. All figures are illustrative.

On Monday morning Alice installs a payment app, chooses Guardian A from those available in her jurisdiction, and completes its onboarding. At 9:00 she deposits 500 USDC; the chain shows an address sending 500 USDC to the Depository, and nothing else. By 9:00 and a few seconds, Guardian A has verified her commitment, opened her account, and issued her a receipt for 500.

At 9:12 she pays Bob 20 USDC for last week's dinner. Bob is served by Guardian B. Alice's app posts the encrypted intent; Guardian A debits her books and posts the cross-Guardian message; Guardian B credits Bob and sends him the notice with Alice's memo. Bob's phone shows the 20, which is final from his perspective, approximately one second after Alice initiated the payment. Since Alice's deposit is still unclaimed, Guardian A advanced the 20 from its working-capital account; the advance is invisible to Alice, and no aspect of her payment depends on her awareness of it.

At 11:00 an epoch closes. Guardian A's proof claims a batch of a few hundred pending deposits (Alice's among them, indistinguishable inside the batch), creates her ledger entry at 500, applies her payment, repays its own advance, and nets the 20 owed to Guardian B against everything else the two exchanged that epoch. The Depository verifies and adopts the new state root, and the public record shows one net figure per Guardian: Guardian A's bucket increased by an amount aggregating hundreds of unrelated claims, payments, and exits, and Guardian B's changed by another. Alice's 500, her 20, and her choice of Guardian A appear nowhere in it.

It is worth enumerating the information held by each party at the end of the day. Guardian A knows Alice's account completely; that is its role. Guardian B knows Bob received 20 through Guardian A, and does not know from whom. Bob knows the payment came from Alice, because she told him so in the memo. The chain shows that some address deposited 500 USDC that morning, and that at 11:00 the Guardians settled to new totals. Months later, Alice becomes dissatisfied with Guardian A's fees and moves her account to Guardian C. No observer, including Guardian A's competitors, can determine that the move occurred.

3.10 What the chain still shows

A precise statement of what remains public is a necessary part of any privacy analysis. The public record retains:

- **Deposits and refunds:** the depositing address, the amount, and the time. The anonymity of the subsequent Guardian assignment is provided by the cohort of deposits claimed alongside it (§3.5); refunds of unclaimed deposits are likewise public.
- **Exits:** the receiving address, the amount, and the time. Withdrawal is possible at any time, and it is publicly visible in every case (§3.8).
- **Bucket totals:** each Guardian's total under guardianship, continuously, by design (P5).
- **Settlement deltas:** one net figure per Guardian per epoch, aggregating deposits claimed, exits paid, and all cross-Guardian traffic. Individual payments are not recoverable from it, with the caveat that an epoch containing few other events provides a small anonymity set (the same caveat applies to deposits).
- **Message traffic:** counts and timing of encrypted messages per Guardian inbox. Guardians may pad their batches to fixed sizes to limit traffic analysis; the ciphertexts themselves reveal nothing.
- **Account counts:** approximate active-account counts per Guardian, from the published commitment updates (§3.4).
- **Settlement timing:** the adaptive cadence of §3.7 reveals when aggregate exposure crossed a threshold, and only in aggregate; a cadence floor limits even that disclosure.

Everything else (the identities of payer and payee, amounts, account balances, and the assignment of users to Guardians) is invisible to every observer except each user's own chosen Guardian; the protocol's privacy claim extends exactly that far.

3.11 Failure modes

An assessment of the design should include its behavior under failure. The table below lists each failure mode, together with its consequence and its backstop.

Failure	Consequence	Backstop
Guardian outage or permanent failure	Confirmations pause; after a bounded number of missed epochs the subtree freezes at the last proven root	Unilateral exits against the frozen root; receipts and the failing Guardian's working capital cover within-epoch credits; every other Guardian settles normally
Guardian censors a user	Payments through that Guardian stop; nothing else does	Permissionless switch or exit; forced inclusion on the message board
Guardian over-issues (books exceed the bucket)	No valid epoch proof exists; the Guardian is delinquent by construction	The solvency identity (§3.4); its working capital
Guardian data breach	Plaintext account data of that Guardian's users may be exposed; funds are unaffected	Privacy insurance and the honeypot (§4, §5); threshold decryption of Guardian traffic (§8)
User loses their device	Nothing is lost	The seed restores keys; history, receipts, and the ledger entry re-derive from the message board (§3.3)
User is coerced to withdraw	The attacker is subject to the user's own exit policy	Tiered delays, approver thresholds, slow policy changes; the hard-exit path remains for the user (§3.8)
Prover unavailability	Settlement is delayed; books and receipts continue	Proving and aggregation are permissionless; any party may compute and submit

Table 1: Failure modes and their backstops.

3.12 Alternatives considered

Four families of prior design shaped this one, and each was considered as a foundation.

On-chain encrypted balances (the Zether lineage) keep every balance on Ethereum as a ciphertext and prove each transfer correct individually. The primitives are clean and well studied. However, per-payment proofs on the base layer cannot carry a payments network's volume, and the model offers no natural locus for a compliance relationship. This design moves per-payment work onto Guardians' books and proves per epoch instead.

Shielded pools and private rollups (the Zcash lineage; Aztec as its most complete expression) offer the strongest privacy available. However, they address a different requirement: they conceal activity from everyone, operators included, while this network's premise is concealment from everyone except one accountable party the user chose. Rebuilding Guardian accountability atop a system engineered to prevent it would conflict with the design intent of both systems, and deploying on such a platform would add the operating assumptions of a comparatively new chain, and bridged assets, to the trust base. Their tooling, however, is directly reusable here (the circuits of this section could reasonably be written in the languages that ecosystem produced), and their note-encryption and viewing-key techniques set the standard from which this design borrows.

Encrypted execution (fully homomorphic encryption chains and trusted-hardware networks) would address these requirements most directly: balances that stay encrypted on chain and are mutated in place. At present the approach is bounded by performance and by trust in decryption committees or hardware vendors. It is the natural long-run direction, and the same trajectory (replacing each Guardian's

single key with a threshold key held by several parties) appears in §8 as an upgrade path that changes no other part of the architecture.

Federated e-cash (the Chaumian lineage, in its modern federated forms) is closest in spirit: instant private payments through a service federation the user chooses. Its structural limit is that the federation custodies the funds. The Depository exists precisely to remove that custody (Guardians here operate accounting they can neither spend from nor freeze beyond a settlement cycle) while keeping the service relationship that makes the system usable and governable.

It is worth noting that the novelty of this design lies in its composition rather than in its components. Commitments, nullifier-style claiming, Merkle state, epoch proofs, and exit delays each have years of production history elsewhere, and the protocol requires no new cryptography. The one component that still involves genuine research questions, threshold decryption of Guardian traffic, is on the roadmap rather than the core design.

4. The Insurance Economy

4.1 The source of funds

A user's deposit does not remain idle. The Depository holds its reserves as a diversified basket of yield-bearing dollar instruments (tokenized treasury funds and similar assets), whose composition is a governed parameter (§8) and public on-chain at every block. Balances remain denominated in dollars and redeemable one-for-one, and the yield funds the operation of the network. In the example figures used throughout this paper, reserves earn a few percent annually, with half the yield paying Guardians and half paying privacy-insurance premiums. There is no protocol token, and no fee is taken from payments; the economics of the network are funded entirely by reserve yield.

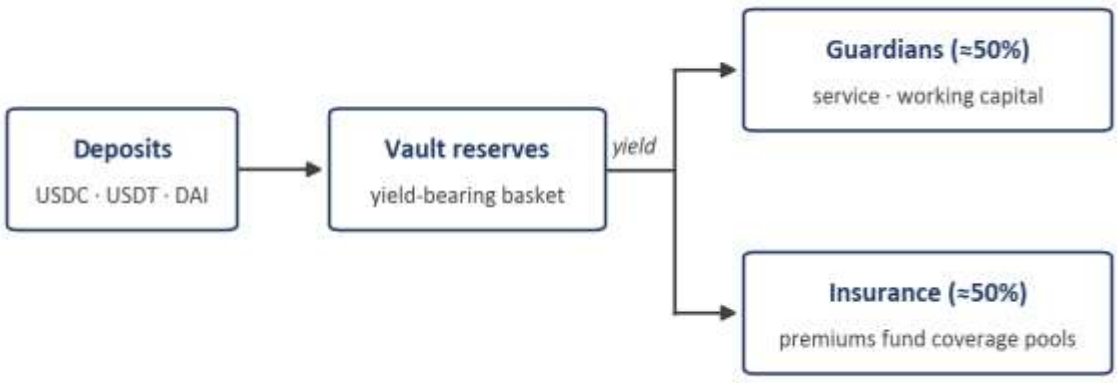

Figure 4: The flow of value, in example figures. Reserve yield funds the Guardians and the privacy insurance.

The reserve carries the risks of its instruments (de-pegs, duration, issuer failure), and those risks are borne by the pool as a whole. Diversification across multiple instruments bounds any single failure, and an impairment, should one occur, is shared pro-rata by all balances rather than falling on the last users to

redeem. This is the structure under which money-market funds have operated for decades, with one difference: the reserve here is visible on-chain at all times and auditable by anyone, block by block. There is no per-user choice of basket. The pool is collective, which keeps it simple, liquid, and inexpensive to operate.

4.2 Coverage pools and underwriters

Each Guardian stands behind an insurance coverage pool, funded from two sources: the Guardian's own co-fund, capital that it forfeits if it fails its users (15 percent of the pool, in the example figures), and underwriting capital drawn from an open market and paid from the premium stream. Any party may underwrite: funds, DAOs, individuals, and professional insurers. Underwriting is opt-in per Guardian, and premiums pay for diligence (§5.7): an underwriter backs the Guardians whose security it has examined, at prices that reflect its findings.

Diversification makes the coverage affordable. Hundreds of Guardians across dozens of jurisdictions fail, when they do fail, for mostly unrelated reasons; a breach in one country does not correlate with a breach in another. Capital can therefore back many pools at once, in the same way that every insurance market since Lloyd's has stretched capital across uncorrelated risks, and a unit of underwriting capital supports several units of user coverage. As a result, the premium stream purchases more protection than any single Guardian could purchase alone.

4.3 The visible quality signal

A user cannot inspect a Guardian's security practices, and the design does not require the user to do so. However, a user can read coverage: the amount that the Guardian's pool would pay in the event of a leak of that user's data (§5). That number is public, binding, and directly comparable across Guardians, which makes it the market's quality signal. First, a Guardian that underinvests in security pays for that choice through higher premiums. Second, a Guardian that cannot attract underwriters cannot advertise coverage. Finally, a new entrant with excellent security and deep coverage can compete from its first day with a long-established incumbent. The capital that a Guardian posts behind its coverage terms therefore performs the role that the reputation of an established brand performs in conventional financial services. This also makes the market more competitive, as brand reputation is subject to economies of scale, whereas the insurance pool is linear with the number of users (as only the per-user payout matters to a user).

4.4 Prospective Guardians

The economics admit a wide field of prospective Guardians. Banks and licensed fintechs already operate compliance functions, and guardianship is adjacent work that adds a revenue line. Merchant networks may be the most interesting entrants: a retail chain that becomes a Guardian earns the yield share on its customers' balances and is already present at the point of sale. It can fund sign-up discounts directly from that share, a distribution channel that every payments network in history has had to purchase at considerable expense. Telecoms bring existing billing relationships, and, where law permits, decentralized operators serve users who prefer not to trust any institution. The obligations are uniform for every entrant: posting the co-fund, publishing coverage terms, securing the data to the standard that §5 enforces, and answering lawful process (§7).

Each element described above (the co-fund, the premiums, the coverage numbers) rests on a single premise: that a breach, when it occurs, is detected and paid, and that no committee determines whether to acknowledge it. That trigger is the subject of the next section.

5. The Honeypot

The protocol's compliance thesis concentrates a significant risk in a single party. A user's own Guardian sees that user's account completely (§3.1): identity, balance, every payment. The architecture withholds that knowledge from every other party, but it cannot prevent the Guardian itself from leaking, selling, or losing it. However, the architecture can ensure that each of those outcomes is revealed under a standing incentive, compensated automatically once proven, and severely costly for the Guardian. That is the role of the honeypot.

Each Guardian's insurance pool (§4) sets aside a standing public bounty, the honeypot, payable by smart contract to anyone who proves possession of that Guardian's confidential dataset. (The term is used in the bounty sense, a reward deliberately made public, rather than in the decoy sense of traditional security practice.) In the example figures used throughout this section, the pool is funded roughly 15 percent by the Guardian's own slashable co-fund, with the remainder underwritten by insurance providers in an open market, and 5 percent of the pool stands as the honeypot.

Data theft is ordinarily covert, deniable, and litigated years after the harm has occurred. Under this design, the same event proceeds differently: the theft becomes public, its occurrence can be proven, settlement occurs in code within a day or two, and the theft is reported by a party who holds the stolen data, since only such a party can make a valid claim.

5.1 What the mechanism must do

Four requirements shape the design.

H1. Detection without adjudication. A breach is proven to a contract and settled by it. Neither a committee, nor an insurer's consent, nor a court is required in the settlement path; those institutions may act afterward, but the payout does not wait for them.

H2. Verification without re-leaking. Proving that private data was stolen must not publish the private data. A naive mechanism (one that shows the contract what was taken) would make every claim a second breach.

H3. Compensation without a victim list. Paying the affected users must not identify them. A public roster of breach victims would itself constitute a disclosure.

H4. Claims without permission. Anyone can claim, pseudonymously, from a fresh address. The design assumes claimants may include criminals, disgruntled insiders, and the Guardian's own contractors, among other parties.

5.2 Ground truth: what "having the data" means

A bounty for possessing a dataset requires an incorruptible definition of the dataset. The protocol supplies one as a side effect of settlement. Every epoch, a Guardian publishes hiding commitments to each ledger

entry it maintains (§3.4), and every message it processes is bound by a commitment posted alongside the ciphertext (§3.6). The Guardian's operating database (account keys, balances, transaction histories, and the commitment randomness that accompanies them) is exactly the set of openings of those on-chain commitments, and the settlement proofs already anchor per-epoch roots for all of it. The phrase "Guardian G's confidential data from epoch X to epoch Y" therefore refers to a canonical, on-chain-committed object rather than to loose wording in an insurance contract, and knowledge of that object is mechanically testable. In other words, what a claimant proves they hold is precisely the database whose theft harms users.

The covered dataset is the protocol's own data. Records a Guardian keeps outside the protocol (identity documents gathered at onboarding, network logs) have no on-chain commitments to test against, and fall to the conventional layer of the insurance rather than to the honeypot (§5.7).

5.3 The claim game

A claim proceeds as a five-move game between a claimant and a contract, with no discretionary step at any point.

1. Claim. From any address (a fresh one, if the claimant prefers), a claim names a Guardian and a scope: a range of epochs, always covering all of that Guardian's accounts rather than a chosen subset. The claim posts a stake, forfeited if the claim fails, and a binding commitment R to the claimant's copy of the scoped dataset, indexed in the same canonical order as the on-chain record.
2. Challenge. Only after R is fixed does the contract draw the challenge: m indices sampled uniformly from the scope's ledger entries and messages (64, as an example figure), using a public randomness beacon.
3. Response. Within a short window (hours, as an example figure), the claimant submits one batched zero-knowledge proof asserting, for each sampled index, that the claimant knows the opening of the on-chain commitment and that it matches position i of the pre-committed R. The proof reveals nothing about the data itself (requirement H2).
4. Verdict. The contract verifies the proof against the per-epoch roots it already holds. If the proof is correct on at least $m - \delta$ of the m samples (δ small, tolerating a copy with minor gaps), the breach is proven. If it falls short, the claim fails and the stake joins the pool.
5. Consequences. Automatically, in the same transaction: the honeypot pays the claimant; the Guardian's co-fund is slashed into the pool; compensation begins for every account active in the scope (§5.6); and the scope is marked breached. The first valid proof takes the bounty; later proofs of the same data pay nothing, so a single breach cannot yield repeated payouts.

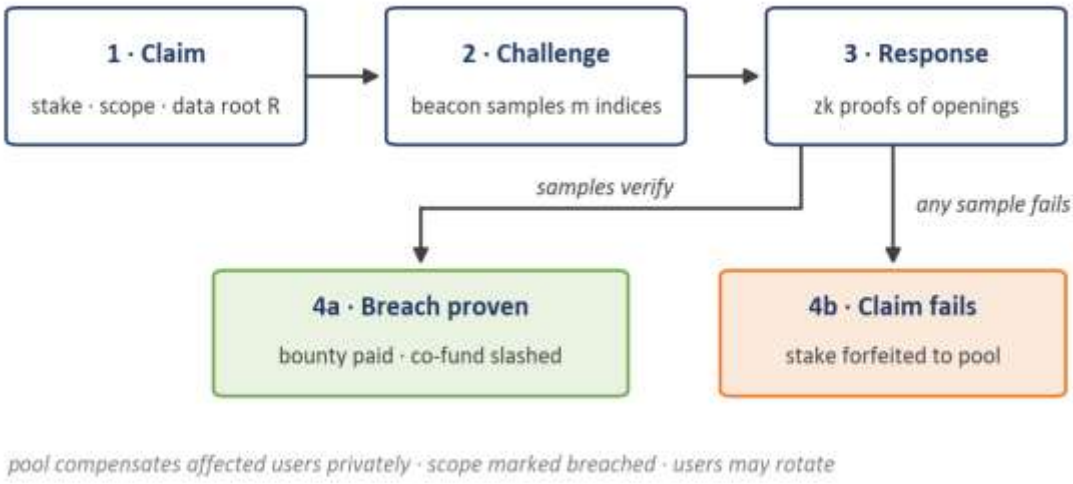

Figure 5: The claim game. The dataset root R is committed before the challenge is drawn, so answers cannot be acquired after the fact.

The pre-committed root R is the element that makes the sampling sound. Without it, a claimant holding nothing could wait for the challenge and then attempt to obtain just the m sampled openings (by bribing a small number of users) inside the response window. Because R is fixed before the draw, answers acquired afterward are useless: they will not match a commitment made in ignorance of the sample. To pass, the claimant must have held substantially the whole dataset before the claim was initiated.

Claims are public from the moment they are posted, and a pending claim against a Guardian is, by design, a source of uncertainty for observers. The game therefore resolves quickly (a day or two end to end, as an example figure), and a false claim fails visibly and forfeits its stake. Observers are likely to learn rapidly that only proven claims are informative.

5.4 Why sampling is proof

The underlying statistics are those that secure decentralized storage networks, applied here to theft rather than to storage. A claimant holding a fraction f of the dataset passes one uniform sample with probability f, and m independent samples with probability f to the power m. At m = 64, a claimant holding half the dataset passes with probability below one in 10^{19}, and a claimant holding 95 percent of it still passes with probability under one in 25. A genuine thief with a complete or near-complete copy passes with high probability: at 98 percent completeness, with the small tolerance δ = 2, the claim succeeds more often than five times in six. In other words, the test distinguishes, with high reliability in both directions, a claimant who holds substantially the whole dataset from one who does not.

The rule that scopes span all accounts is what converts this arithmetic into a security property. Every user knows the openings of their own entries (statelessness guarantees this, §3.3), so fragments of the dataset are always legitimately in circulation. However, one user, or even a colluding hundred, holds only a small share of the sampling frame, and small shares fail the sampling with overwhelming probability. To fabricate a breach where none occurred, a coalition would need essentially the entire customer base. A coalition of that size would in fact hold the complete dataset, and the situation becomes one of a customer

base defrauding its own insurers, which is a scenario underwriters price rather than one the protocol can prevent (§5.7).

5.5 The economics of never wanting to win

Consider the example pool: the Guardian's slashable co-fund at 15 percent of it, and the honeypot at 5.

The Guardian itself can always pass the sampling, since it holds the data. It never profits by doing so. A self-claim through a proxy collects the honeypot, forfeits the larger co-fund, and eliminates the fee stream and client base that made the guardianship business viable. Provided that the honeypot is smaller than what the Guardian stands to lose (the example figures leave a threefold margin before counting the business itself), self-breach is strictly value-destroying, including for a Guardian already winding down, since an orderly exit returns the co-fund intact.

Employees present the opposite case, and the one the mechanism is designed to address. An insider owns none of the co-fund; for an insider, the honeypot is a standing offer with no offsetting loss. A Guardian must therefore operate so that no employee (and no plausible coalition of employees) can assemble the complete dataset, which in practice means encrypting storage under keys no single person holds, sharding access, and placing alarms on data movement. As a result, the party with the strongest incentive to protect the users' data becomes the Guardian itself, and the honeypot funds that protection continuously, without the involvement of an auditor.

The subtlest effect concerns the covert market. Stolen data escapes the honeypot only if every party who ever handles it declines the standing bounty. A buyer of stolen data can take delivery and then claim the bounty, anonymously and profitably, slashing the seller's co-fund in the process. Every covert sale must therefore survive a standing offer of payment for defection. In addition to detecting breaches, the honeypot undermines the mutual trust that transactions in stolen data require, and this trust problem is more difficult for the parties involved to overcome than encryption.

For the claimant, the bounty is designed to be more attractive than a covert sale: it pays without identity, without negotiation, and without the trust problem a buyer would otherwise present. The honeypot is therefore likely to act as a floor price on the dataset, in the sense that data worth less than the bounty is more profitably claimed against the pool than sold covertly. The exposure is also bounded by retention: a claimant can prove possession only of what the Guardian's database contained when it was taken, so a Guardian that retains less recorded history has less that can be stolen and less that can be claimed against it. The honeypot therefore prices data retention directly, and rewards data minimization within whatever retention the law requires. The effect is prospective only: deleting data cannot immunize a copy that has already left the Guardian, because a claim is proven by whoever holds the copy, against the chain's own commitments, and the Guardian has no role in the verification.

It is worth noting that nothing in the mechanism imposes a deadline on claims. A Guardian's liability for a leaked scope therefore persists for as long as its pool stands, which is the permanent form of accountability the design intends. One consequence is that a lone holder of stolen data faces no time pressure to come forward, although the moment a second party holds the copy, each holder risks the other claiming first. If prompt disclosure were desired as a property in its own right, a bounty that decays with the age of the claimed scope would be a straightforward design option; this paper leaves the bounty flat.

5.6 After a proof

Every consequence of a proven claim executes in code, in a fixed order.

- **The bounty.** The honeypot pays to whatever address claimed. No identity is requested; requirement H4 is maintained through the final step.
- **The slash.** The Guardian's co-fund moves into the pool, joining the insurers' capital available for compensation.
- **Compensation.** Every account active in the breached scope is compensated from the pool under the policy's terms, credited through the ledger itself. The pool's funds enter the Depository and reach affected accounts as ordinary private credits over the following epochs, through whichever Guardian serves each user by then. No roster of victims is ever published (requirement H3); the compensation is as private as the payments were.
- **The mark.** The scope is marked breached, and later proofs of overlapping scope pay nothing, so a single theft pays a single bounty.
- **Rotation.** The stolen dataset begins depreciating immediately. Affected users can move to a new Guardian under fresh account identifiers using the ordinary switching machinery (§3.8), cooperatively, or unilaterally if confidence has been lost. As a result, the stolen data describes accounts that, within days, no longer transact.
- **Repricing.** The effect of a proven breach on the Guardian's premiums, its co-fund requirements, and its viability is addressed in §4.

5.7 What the honeypot cannot see

The mechanism has blind spots, and they are stated below.

- **Knowledge that is not the dataset.** An analyst's memory, a verbal disclosure, or a summary of one user's habits cannot be sampled. Such harms fall to the Guardian's contract terms, to privacy law, and to the conventional layer of the insurance.
- **Data without commitments.** Identity documents, network logs, and other records held outside the protocol have no commitments to test against. Extending commitments to such records is possible in principle and is deliberately excluded from the core design.
- **Small thefts.** A copy far below completeness fails the sampling, by the same property that blocks fabricated claims. The honeypot therefore detects the catastrophic case: the theft of substantially the whole dataset over some period. The theft of a small fraction of the data cannot be proven through the claim game, so any compensation for partial exposure would have to come from the pool's ordinary policy terms (§4) rather than from the automatic mechanism.
- **Manufactured Guardians.** A fabricated Guardian with fabricated users could pass its own sampling and drain a pool. The protocol gates the honeypot on a pool's age and deposit history; the decisive defense is that underwriting is opt-in. Insurers choose which Guardians to back, and diligence is what premiums pay for (§4).
- **Provenance.** A copy that left the Guardian through lawful process, and leaked from its recipient, is possession all the same. How fault and slashing should apportion in such cases is a genuine open question, and this paper leaves it open (§8).

Within those limits, the guarantee is exact. Any complete copy of a Guardian's dataset, wherever it resides, is a liability to whoever holds it, and anyone to whom the holder shows the copy gains an incentive to claim the bounty against them.

6. Using the Network

6.1 Everyday use

From the user's perspective, the mechanisms described in §3 reduce to a short sequence of steps. A user installs a payment app, selects among the Guardians available in their jurisdiction (with coverage, fees, and protections displayed side by side), completes that Guardian's onboarding process, and deposits funds from any wallet or exchange; a mature network adds deposits made directly from bank rails. From that point, payments to any other participant on the network confirm in about a second and carry no cost at the point of use (§6.4). The entire account is controlled by a single seed phrase, so the loss of a phone costs only the time required to restore access (§3.3). In addition, the Guardian can be changed in much the same way a phone number is moved between carriers, including over the objection of the previous Guardian (§3.8). Users do not handle gas, do not sign raw transaction data, and do not encounter an address unless they leave the network; Guardians perform the chain interaction on their behalf, which is one of the services the yield share funds.

6.2 User security policies

Permissionless exit is the mechanism that guarantees the network's neutrality, and it is also the path a thief would use, so each user sets their own exit policy, which is enforced by the protocol itself and cannot be removed by the user's Guardian (§3.8). In example configurations, the policy space is structured as follows: small withdrawals complete quickly; larger withdrawals are subject to delays scaled to the amount; and the largest withdrawals require approvals from a set of parties the user has chosen (family members, a lawyer, a service, and optionally the Guardian), at thresholds such as one-of-three for medium sums and two-of-three for large sums. Policy changes take effect slowly, so that compromising a phone does not compromise the policy. Beneath these layers is the hard exit, a long-delay path that requires no approvals at all, which ensures that a user abandoned by every counterparty can still withdraw their funds. The same approver machinery also serves as a recovery mechanism: keys can be restored socially, without any Guardian's permission.

6.3 Bounded reversibility

Between Guardians, payments are final at confirmation (§3.7). For consumers, finality is adjustable within published limits: within a stated dispute window, a payment can be unwound (as a new, co-signed offsetting entry, evidence-based and jurisdiction-specific) under the protection policies of the Guardians involved. Different Guardians will offer different windows and standards, merchants advertise theirs, and users choose accordingly. Two constraints preserve the integrity of this reversibility. First, reversals are policy applied above the ledger; the canonical record itself is never mutated and only moves forward. Second, exit delays exceed dispute windows (§3.8), so value under dispute cannot leave the network before the dispute resolves. The purpose of bounded reversibility is deterrence, and any reassurance it

offers users is secondary. A fraudulent scheme, a robbery, or a coerced transfer inside the network can be undone within the window, and as a result the network is an unattractive venue for theft.

6.4 Fees

We would expect ordinary payments to be free at the point of use, funded by the reserve yield in much the same way card rewards are funded by interchange, except that no interchange is charged. Guardians compete above the free tier, offering services such as instant external payouts, merchant tooling, invoicing, credit, and enhanced protection services.

7. Compliance

7.1 One protocol, many regimes

Compliance in this design belongs to Guardians. The protocol itself has no compliance policy of its own. A helpful analogy is the internet's transport layer, which has no content policy, while the services running on it answer to their regulators. Concretely, this works as follows. First, jurisdictions publish, or recognize registries of, the Guardians permitted to serve their residents. Second, payment applications (native mobile apps, of which there may be several compatible implementations) enforce the applicable list at the user-experience layer. Finally, Guardians implement their jurisdiction's identity, recordkeeping, and reporting standards as their own policy. The protocol carries whatever those policies require (including the encrypted originator-information slot of §3.6 for travel-rule regimes) and mandates none of it globally. In other words, one protocol serves many regimes without imposing a one-size-fits-all standard, whether the weakest or the strictest, on any jurisdiction.

7.2 The audit model

A user's Guardian can respond to lawful process concerning that user in the same manner as a bank, providing identity information to its onboarding standard, transaction records, and supporting evidence. Two properties distinguish these records from a bank's. First, they are tamper-evident: a Guardian's history must open commitments that the chain fixed long ago (§3), so records cannot be retroactively altered by any party, including the Guardian. Second, they are precisely scoped: a Guardian can answer about its own users and is unable, as a matter of architecture rather than policy, to answer about anyone else's, because it never held the data (§3.4). As a result, lawful access flows through the party accountable for it, under the law that binds it. It is worth noting that because no data exists at the protocol level, there is no protocol-level mechanism through which such access could be granted.

7.3 Nothing to ban

Two properties establish that no government needs to ban this network. First, entry is screened where jurisdictions want screening: Guardians decide whom to onboard, under their own law (§3.5). Second, withdrawals are public: funds leave the network only onto Ethereum's transparent ledger, at a visible address and amount, while a change of Guardian keeps the account inside the network, under another Guardian's compliance regime (§3.8). The protocol guarantees that a user can always leave, and it makes every withdrawal visible. In other words, anyone who exits the private layer surfaces on a transparent

one, so the network cannot function as a repository in which value disappears permanently from public view.

Banning the protocol would therefore gain a government nothing that regulating Guardians does not already provide with more precision.

8. Governance and Roadmap

8.1 Governance

The settlement layer is designed to be boring. The properties that make the network trustless are immutable: the solvency identity, the proof requirements, and every exit right. No authority, however constituted, can move funds, alter proven state, or block an exit. What remains governable is a short list of parameters: the deposit lock window, epoch cadence bounds, sampling sizes, the honeypot and co-fund fractions, and the reserve basket composition. Parameter changes execute behind long timelocks with public notice, always longer than the hard-exit path, so that any user who objects to a pending change can leave, with everything they own, before the change takes effect. In other words, the exit right functions as the network's constitution. The body that proposes parameter changes is deliberately minimal and will be specified with the implementation.

8.2 Roadmap

These are upgrade paths for whoever implements the network, not commitments by the author.

- **Threshold Guardians.** The largest residual risk in the design is the compromise of a single Guardian's key. The upgrade path replaces each Guardian's key with a threshold key held across several independent parties, so that no single compromise (a hack, an insider, or a seizure) decrypts anything. Jurisdictions could even mandate committee structures for lawful access (for example, requiring two licensed parties to cooperate). Nothing else in the architecture changes.
- **Multi-Guardian routing.** Routing payments across multiple Guardians, so that even a user's own Guardian observes less than the whole of each transaction. This is the shared-key counterpart to threshold decryption.
- **Bank-rail onboarding.** Deposits and withdrawals directly from traditional bank accounts and payment applications, so that onboarding does not require an account at a cryptocurrency exchange.
- **Provenance adjudication.** The open question of §5.7, namely how fault should be apportioned when lawfully disclosed data leaks from its recipient. Approaches under consideration include decentralized adjudication.
- **Identity commitments.** The optional extension bringing off-protocol records, such as onboarding documents, under the honeypot's coverage (§5.7).

Payments are only the base layer of financial life. A network that holds payment history under user-controlled disclosure can support reputation; reputation, in turn, can support credit; and Guardians with merchant roots have extended credit since department stores invented the charge card. Private, uncollateralized lending against portable credit reputation is the natural next step, and it is the subject of a separate paper.

9. Conclusion

If we want mass adoption, expecting every government to change their laws for Ethereum is a non-starter. Mass adoption of payments requires native mobile apps. And governments can easily demand that Apple and Google ban a non-compliant app in their country.

The vast majority of people are not going to run Tor with a VPN, while storing their shielded-wallet keys on an encrypted USB just to make private payments.

Instead, they'll look to PayPal, or Visa, or Stripe, and say 'good enough for me, whatever.'

And we could have offered them so much better.

We *can* offer them so much better. But it has to check all the boxes.

That's the intent of this design. Do maximum good. Offer the most privacy to the most people we can. All the while driving Ethereum mass adoption.

And then we live to fight another day, so that we can offer even *more* privacy in the future.

But let's keep that part between us. Privacy being important and all.